

5 zasad dobrych testów jednostkowych

Ten dokument opisuje podstawowe zasady tworzenia **dobrych testów jednostkowych**. Zasady te mają pomóc w pisaniu testów, które **wspierają rozwój systemu**, a nie są jedynie formalnym obowiązkiem.

1. Test opisuje zachowanie, nie implementację

Dobry test odpowiada na pytanie:

Co system powinien zrobić w tej sytuacji?

Test:

- nie sprawdza metod prywatnych
- nie zależy od struktury kodu
- nie psuje się przy refaktoryzacji, jeśli zachowanie się nie zmieniło

Testy są kontraktem zachowania systemu.

2. Test jest czytelny jak zdanie

Po przeczytaniu testu powinno być jasne:

- jaki scenariusz jest testowany
- w jakich warunkach
- jaki jest oczekiwany rezultat

Dlatego:

- stosuj czytelne nazwy testów
- używaj struktury Arrange / Act / Assert
- unikaj „magicznych” wartości bez kontekstu

Test to dokumentacja zachowania systemu.

3. Test jest deterministyczny

Ten sam test:

- zawsze daje ten sam wynik
- niezależnie od czasu, środowiska czy kolejności uruchomienia

Unikaj w testach:

- `DateTime.Now`
- dostępu do sieci
- plików
- losowości

Jeśli test czasem przechodzi, a czasem nie —
to nie jest test, tylko loteria.

4. Jeden test = jeden scenariusz

Dobry test:

- sprawdza jedno zachowanie
- ma jedną przyczynę porażki
- jasno komunikuje, co poszło nie tak

Jeśli test nie przechodzi, powinieneś od razu wiedzieć:

dlaczego

5. Test daje odwagę do zmiany

Najważniejsza zasada:

Dobry test ułatwia zmianę kodu.

Testy powinny:

- zachęcać do refaktoryzacji
- pozwalać upraszczać kod
- chronić przed regresją

Jeśli boisz się zmieniać kod mimo testów,
to znaczy, że **testy jeszcze nie spełniają swojej roli.**

Podsumowanie

- Testy są **wsparciem**, nie obowiązkiem
- Testy nie są metryką jakości
- Liczy się czytelność i intencja, nie ilość

**Dobry test pomaga zrozumieć zachowanie systemu.
Jeśli tego nie robi – to tylko kolejny kod.**